Visitor Management System

Solution Architecture, Problem Analysis & Technology Stack

Prepared for: UI Prototype Review

Date: 12 April 2026

1. Executive Summary

This document presents a comprehensive solution architecture for the Visitor Management System (VMS) prototype. It details the problems identified in the original VMS specification, the solutions implemented in the interactive prototype, and the complete technology stack chosen to support a production-ready system. The VMS prototype demonstrates all core visitor lifecycle flows — from check-in and webcam capture through employee approval and dashboard analytics — using a modern, enterprise-grade UI.

The system is designed to serve corporate offices, educational institutions, hospitals, and any high-traffic facility that requires structured, auditable visitor access control. The prototype covers all user touchpoints: visitors, receptionists, security administrators, and employees.

Scope of This Document

- Problem identification from original VMS requirements
- Solution design for each identified problem
- Complete frontend and (planned) backend technology stack
- Architecture overview of the prototype implementation
- Future roadmap for moving from prototype to production

2. Problem Analysis – Identified Challenges

The following problems were identified from the VMS requirements document. Each problem is analyzed with its operational impact and the corresponding solution approach applied in the prototype.

01

Manual, Paper-Based Visitor Registration

Impact: Slow check-in, illegible records, lost logs, administrative burden on front-desk staff.

' Solution: Digital check-in form with structured fields (Name, Aadhaar, Phone, Email, Purpose, Meet With). Data stored in state/database instantly.

02

No Identity Verification Mechanism

Impact: Security risk from unverified visitors; inability to trace incidents.

' Solution: Aadhaar (National ID) capture field during check-in; webcam photo capture for visual identity; visitor IDs auto-generated (VIS-001, VIS-002…).

03

No Real-Time Approval Workflow for Employees

Impact: Employees unaware of visitors until they arrive; no structured approve/reject mechanism.

' Solution: Approval Page with live status management. Employees can approve or reject visitor requests; buttons disable after action to prevent duplicate actions.

04

Lack of Visitor Traffic Visibility for Management

Impact: No insights into peak hours, visit purposes, or overall facility usage.

' Solution: Dashboard with metric cards (Total Visitors, Today's Visitors, Check-In/Out counts) and charts — bar chart (7-day traffic) and pie chart (purpose breakdown).

05

No Appointment / Pre-Visit Booking System

Impact: Walk-in-only model causes congestion; employees cannot prepare for scheduled visitors.

' Solution: Schedule Appointment page with a full booking form (name, email, date picker, purpose, host). Appointments tracked as pre-registered visitors.

06

Returning Visitors Must Re-Register Every Time

Impact: Friction for frequent visitors; data entry errors; poor visitor experience.

' Solution: Previous Visits page allows lookup by mobile number, retrieving all historical visit records instantly.

No Secure, Role-Based Admin Access

Impact: Any person could access the system; no separation between visitor-facing and admin-facing

flows.

' Solution: Login Page with credential form. Sidebar navigation only accessible after login context. Admin and receptionist flows are separated in the UI.

3. Solution Architecture

The prototype implements a component-based single-page application (SPA) architecture. All state is managed client-side using React Context, simulating the data flows that a production backend would handle. The architecture is designed so that each client-side mock can be replaced 1:1 with a real API endpoint.

3.1 System Modules

Home Module	Entry portal for visitors. Routes to Check-In, Schedule Appointment, or Previous Visits.
Check-In Module	Structured multi-field form (react-hook-form + Zod validation). Adds visitor record to global state.
Webcam Module	Simulated camera capture interface. In production: integrates with device camera API (MediaDevices.getUserMedia).
Dashboard Module	Admin analytics view. Recharts bar & pie charts with KPI metric cards. All data from VisitorContext.
Approval Module	Live approve/reject workflow. Status badges update in real-time. Buttons disabled after action.
Appointment Module	Pre-visit booking form. Date picker, purpose, host selection. Submission adds to pending appointments.
Previous Visits Module	Returning visitor lookup by phone number. Displays full visit history table.
Login Module	Credential entry form. Triggers authenticated session context (UI only in prototype; JWT-based in production).

3.2 State Management Flow

VisitorContext (React Context API) acts as the global data store for the prototype:

- addVisitor(visitorData) — called on Check-In form submit
- approveVisitor(id) — called on Approval page Approve button
- rejectVisitor(id) — called on Approval page Reject button
- Mock array initialized with 8 pre-seeded visitor records covering all status states

In production, each context action would be replaced by an API mutation (POST/PATCH) with TanStack Query cache invalidation.

4. Technology Stack

The following technologies are selected based on industry best practices for modern enterprise web applications. The stack prioritises developer productivity, type safety, accessibility, and scalability.

4.1 Frontend Stack

Framework	**React 18 + Vite 5** Component-based SPA. Vite provides instant HMR, fast builds, and native ESM.
Language	**TypeScript 5.9** Static typing prevents runtime errors; improves maintainability and IDE support.
Styling	**Tailwind CSS 4 + shadcn/ui** Utility-first CSS. shadcn/ui provides accessible, composable Radix UI components.
Routing	**Wouter 3** Lightweight, hook-based client-side routing. Respects the Vite BASE_PATH prefix.
Forms	**React Hook Form + Zod** Performant uncontrolled forms with schema-based validation. Zod provides end-to-end type safety.
Charts	**Recharts 2** Composable charting library built on D3. Bar chart (daily traffic) and Pie chart (purpose breakdown).
State	**React Context + useState** Prototype-level global state. Production: TanStack Query for server state, Zustand for UI state.
Icons	**Lucide React** Clean, consistent icon library with tree-shaking. 1000+ icons optimised for React.
Animation	**Framer Motion** Production-grade animation library. Used for page transitions and micro-interactions.

4.2 Backend Stack (Production Plan)

Runtime	**Node.js 24 + Express 5**

Non-blocking I/O. Express 5 with async error handling built in.

Database

PostgreSQL + Drizzle ORM

Relational DB for structured visitor records. Drizzle provides type-safe queries and migrations.

Validation

Zod (shared schema)

Zod schemas shared between frontend and backend via monorepo lib. Single source of truth.

API Design

OpenAPI 3.1 + Orval

Contract-first API. Orval generates typed React Query hooks from the OpenAPI spec automatically.

Auth

Session-based (JWT + cookies)

Secure HTTP-only cookies. Role separation: visitor, receptionist, admin, employee.

File Storage

Replit Object Storage / S3

Webcam captures stored as JPEG/PNG blobs. Referenced by visitor record ID.

4.3 Infrastructure & Tooling

Monorepo	**pnpm Workspaces** Single repo with packages: api-server, vms-ui, api-spec, db, api-client-react.
Package Mgr	**pnpm 10** Fast, disk-efficient. Shared dependency catalog via pnpm-workspace.yaml.
Deployment	**Replit Deployments** One-click cloud deployment. Static frontend + Node.js backend. HTTPS included.
Logging	**Pino + pino-http** Structured JSON logging. Request logging middleware auto-attached to Express.
Build	**esbuild (server) + Vite (client)** Sub-second builds. esbuild bundles CJS for Node; Vite bundles ESM for browser.

5. Pages, Routes & Features Implemented

Route	Page Name	Purpose	Key Features
/	**Home**	Entry hero page	3 action cards, navigation to all flows
/login	**Login**	Admin/staff authentication	Credential form, role-based access intent
/checkin	**Check-In**	Visitor registration	8-field validated form, Aadhaar, Purpose, Host
/webcam	**Webcam Capture**	Photo identity capture	Simulated camera preview, Capture confirmation
/dashboard	**Dashboard**	Analytics overview	5 KPI cards, 7-day bar chart, purpose pie chart, visitor table
/approval	**Approval**	Request management	Pending/Approved/Rejected stats, approve/reject per row
/schedule	**Schedule Appt.**	Pre-visit booking	Date picker, host selection, success confirmation
/previous-visits	**Previous Visits**	Returning visitor lookup	Mobile number search, visit history table

6. Data Model – Visitor Record

Each visitor record stored in VisitorContext (and planned PostgreSQL table) contains the following fields:

Field	Type	Description
id	string	Auto-generated unique ID (e.g., VIS-001)
name	string	Full name of the visitor
aadhaar	string	National ID number (12 digits) — masked in display
phone	string	Mobile number — used for returning visitor lookup
email	string	Email address for notification sending
gender	string	Male / Female / Other
purpose	string	Meeting / Interview / Delivery / Personal / Other
meetWith	string	Employee name the visitor intends to meet
status	enum	pending | approved | rejected
photoUrl	string?	URL of webcam capture (null until captured)
date	string	Visit date (YYYY-MM-DD)
time	string	Check-in time (HH:MM AM/PM)
checkoutTime	string?	Check-out time (null if still on premises)

7. Production Roadmap – Prototype !' Live System

The prototype establishes the complete UI surface. The following roadmap outlines the steps to convert this prototype into a fully operational, production-grade Visitor Management System.

Phase 1

Backend API

Build Express 5 REST API. Implement all visitor CRUD endpoints. Connect to PostgreSQL via Drizzle ORM. Deploy on Replit.

Phase 2

Authentication

Implement JWT-based login with HTTP-only cookies. Add role middleware (admin, receptionist, employee). Protect all dashboard routes.

Phase 3

Real Webcam Integration

Replace fake camera preview with MediaDevices.getUserMedia(). Capture JPEG frame. Upload to object storage. Link to visitor record.

Phase 4

Email Notifications

Integrate SendGrid or Nodemailer. Send email to employee on visitor check-in. Send approval/ rejection email to visitor.

Phase 5

Aadhaar / ID Verification

Optional: Integrate government eKYC API for Aadhaar validation. Add OTP-based mobile verification.

Phase 6

Visitor Badge Printing

Generate printable visitor badge PDF on approval. Include QR code with visitor ID for gate scanning.

Phase 7

Real-Time Notifications

Add WebSocket (Socket.io) for live approval updates. Employee receives browser notification on visitor arrival.

Phase 8

Reports & Export

Add date-range filtered visitor reports. Export to PDF/CSV. Department-wise analytics for management dashboards.

8. Summary – Problems vs. Solutions Matrix

Quick reference matrix mapping each identified problem to its solution and the technology used:

Problem	Solution Implemented	Technology U
Manual registration	Digital check-in form	React Hook Form + Zod

No identity verification

Aadhaar field + Webcam capture

No approval workflow

Approval page with approve/reject

No visibility / analytics

Dashboard with KPI cards + charts

Recharts (Bar + Pie)

No appointment booking

Schedule Appointment page

Re-registration friction

Previous Visits lookup by phone

No access control

Login page + role-based routing

JWT + HTTP-only cookies (Phase 2)

No email notifications

Planned email on approval events

SendGrid / Nodemailer
(Phase 4)

Visitor Management System

Solution Architecture & Technology Stack Report

This prototype demonstrates a complete, production-aligned architecture for managing visitor access in modern enterprise facilities.

Built with React 18 · Vite 5 · Tailwind CSS 4 · TypeScript 5.9
shadcn/ui · React Hook Form · Zod · Recharts · Wouter · Framer Motion